

Separate Stack Programs Cpp

1. Stack Implementation Using Array

```
#include<iostream>
using namespace std;

class Stack{
public:
    int* arr;
    int size;
    int top;

    Stack(int capacity){
        arr = new int[capacity];
        size = capacity;
        top = -1;
    }

    void push(int val){
        if(top == size-1){
            cout<<"Stack Overflow"<<endl;
        }
        else{
            top++;
            arr[top] = val;
        }
    }
};

int main(){
    Stack s(5);
    s.push(10);
    return 0;
}
```

2. Two Stacks in One Array

```
#include<iostream>
using namespace std;

class Stack{
public:
    int* arr;
    int size;
    int top1;
    int top2;

    Stack(int capacity){
        arr = new int[capacity];
        size = capacity;
        top1 = -1;
        top2 = size;
    }
};

int main(){
    Stack s(5);
    return 0;
}
```

3. Reverse String Using Stack

```
#include<iostream>
#include<stack>
using namespace std;

int main(){
    stack<char> s;
    string name = "amitwala";

    for(int i=0; i<name.length(); i++){
        s.push(name[i]);
    }

    while(!s.empty()){
        cout<<s.top();
        s.pop();
    }

    return 0;
}
```

4. Insert Element at Bottom of Stack

```
#include<iostream>
#include<stack>
using namespace std;

void insertAtBottom(stack<int>& s, int value){
    if(s.empty()){
        s.push(value);
        return;
    }

    int topElement = s.top();
    s.pop();

    insertAtBottom(s, value);

    s.push(topElement);
}
```

5. Reverse Stack Using Recursion

```
#include<iostream>
#include<stack>
using namespace std;

void reverseStack(stack<int>& s){
    if(s.empty()){
        return;
    }

    int topElement = s.top();
    s.pop();

    reverseStack(s);

    s.push(topElement);
}
```